DATA STRUCTURES AND ALGORITHMS

DSA LAB RECORD

Name : K.Mokshitha

Register Number : 11259A283

Section : S9

Subject : Data structure and algorithm (DSA)

DEPARTMENT OF COMPUTER SCIENCE / ENGINEERING

EXPERIMENT 1: Array Operations

AIM

To perform insertion, deletion, traversal and searching operations on a one-dimensional array.

ALGORITHM

1. Start and read the number of elements and array elements.

2. Display the menu: insert, delete, traverse, search or exit.

3. For insertion, read the position and value, shift elements to the right and insert the value.

4. For deletion, read the position and shift the following elements one position to the left.

5. For traversal, display all elements. For searching, compare the key with each element.

6. Repeat until the user selects exit, then stop.

PROGRAM (C)

```
#include <stdio.h>
#define MAX 100

int main(void) {
    int a[MAX], n, ch, pos, value, i, key, found;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter elements: ");
    for (i = 0; i < n; i++) scanf("%d", &a[i]);

    do {
        printf("\\n1.Insert 2.Delete 3.Traverse 4.Search 5.Exit\\n");
        scanf("%d", &ch);
        switch (ch) {
        case 1:
            if (n == MAX) { printf("Array is full\\n"); break; }
            printf("Position and value: ");
            scanf("%d%d", &pos, &value);
            if (pos < 1 || pos > n + 1) { printf("Invalid position\\n"); break; }
            for (i = n; i >= pos; i--) a[i] = a[i - 1];
            a[pos - 1] = value; n++;
            break;
        case 2:
            printf("Position to delete: "); scanf("%d", &pos);
            if (pos < 1 || pos > n) { printf("Invalid position\\n"); break; }
            for (i = pos - 1; i < n - 1; i++) a[i] = a[i + 1];
            n--; break;
        case 3:
            for (i = 0; i < n; i++) printf("%d ", a[i]);
            printf("\\n"); break;
        case 4:
            printf("Element to search: "); scanf("%d", &key);
            found = 0;
            for (i = 0; i < n; i++)
                if (a[i] == key) { printf("Found at position %d\\n", i + 1); found = 1; break; }
            if (!found) printf("Not found\\n");
            break;
        }
    } while (ch != 5);
    return 0;
}
```

SAMPLE OUTPUT

Example: Input elements 10 20 30; insert 15 at position 2; traverse. Output: 10 15 20 30

RESULT

Thus, insertion, deletion, traversal and searching operations on an array were implemented and executed successfully.

EXPERIMENT 2: Structure Operations

AIM

To define a structure and read, display and process student details using an array of structures.

ALGORITHM

1. Define a structure Student containing roll number, name and marks.

2. Read the number of students.

3. Use a loop to read each student's details.

4. Use another loop to display the details and calculate the class average.

5. Stop.

PROGRAM (C)

```
#include <stdio.h>

struct Student {
    int roll;
    char name[30];
    float marks;
};

int main(void) {
    struct Student s[50];
    int n, i;
    float total = 0;
    printf("Enter number of students: ");
    scanf("%d", &n);
    for (i = 0; i < n; i++) {
        printf("Enter roll, name and marks: ");
        scanf("%d %29s %f", &s[i].roll, s[i].name, &s[i].marks);
        total += s[i].marks;
    }
    printf("\\nRoll\\tName\\tMarks\\n");
    for (i = 0; i < n; i++)
        printf("%d\\t%s\\t%.2f\\n", s[i].roll, s[i].name, s[i].marks);
    if (n > 0) printf("Average = %.2f\\n", total / n);
    return 0;
}
```

SAMPLE OUTPUT

Example input: 2 students — 101 Arun 85.5; 102 Banu 90.0. Output displays both records and Average = 87.75.

RESULT

Thus, student records were stored, accessed and displayed successfully using structures.

EXPERIMENT 3: Stack Using Array

AIM

To implement stack operations (push, pop and display) using an array.

ALGORITHM

1. Initialize TOP to -1.

2. For PUSH, check overflow. If not full, increment TOP and insert the item.

3. For POP, check underflow. If not empty, display the top item and decrement TOP.

4. For DISPLAY, print elements from TOP down to 0.

5. Repeat the menu until exit.

PROGRAM (C)

```
#include <stdio.h>
#define MAX 50

int main(void) {
    int stack[MAX], top = -1, ch, x, i;
    do {
        printf("\\n1.Push 2.Pop 3.Display 4.Exit\\n");
        scanf("%d", &ch);
        if (ch == 1) {
            if (top == MAX - 1) printf("Stack overflow\\n");
            else { printf("Enter value: "); scanf("%d", &x); stack[++top] = x; }
        } else if (ch == 2) {
            if (top == -1) printf("Stack underflow\\n");
            else printf("Deleted: %d\\n", stack[top--]);
        } else if (ch == 3) {
            if (top == -1) printf("Stack is empty\\n");
            else { for (i = top; i >= 0; i--) printf("%d ", stack[i]); printf("\\n"); }
        }
    } while (ch != 4);
    return 0;
}
```

SAMPLE OUTPUT

Example: PUSH 10, PUSH 20, DISPLAY. Output: 20 10. POP displays Deleted: 20.

RESULT

Thus, PUSH, POP and DISPLAY operations were implemented successfully using an array-based stack.

EXPERIMENT 4: Conversion of Infix to Postfix

AIM

To convert an infix expression into its equivalent postfix expression using a stack.

ALGORITHM

1. Scan the infix expression from left to right.
2. If the symbol is an operand, add it to the postfix expression.
3. If it is '(', push it onto the stack. If it is ')', pop operators until '(' is found.
4. For an operator, pop operators with higher or equal precedence (except '('), then push the current operator.
5. After scanning, pop all remaining operators and append them to the output.
6. Display the postfix expression.

PROGRAM (C)

```
#include <stdio.h>
#include <ctype.h>
#define MAX 100

char st[MAX];
int top = -1;

void push(char c) { st[++top] = c; }
char pop(void) { return st[top--]; }
int prec(char c) {
    if (c == '^') return 3;
    if (c == '*' || c == '/') return 2;
    if (c == '+' || c == '-') return 1;
    return 0;
}

int main(void) {
    char in[MAX], post[MAX], c;
    int i, j = 0;
    printf("Enter infix expression (single-letter/digit operands): ");
    scanf("%99s", in);
    for (i = 0; in[i] != '\\0'; i++) {
        c = in[i];
        if (isalnum((unsigned char)c)) post[j++] = c;
        else if (c == '(') push(c);
        else if (c == ')') {
            while (top != -1 && st[top] != '(') post[j++] = pop();
            if (top != -1) pop();
        } else {
            while (top != -1 && st[top] != '(' &&
                   (prec(st[top]) > prec(c) ||
                   (prec(st[top]) == prec(c) && c != '^')))
                post[j++] = pop();
            push(c);
        }
    }
    while (top != -1) post[j++] = pop();
    post[j] = '\\0';
    printf("Postfix: %s\\n", post);
    return 0;
}
```

SAMPLE OUTPUT

Input: A+B*C. Output: ABC*+

RESULT

Thus, the given infix expression was converted into postfix form using a stack.

EXPERIMENT 5: Postfix Evaluation

AIM

To evaluate a postfix expression using a stack.

ALGORITHM

1. Initialize an empty stack and scan the postfix expression from left to right.

2. If the token is a number, push its value onto the stack.

3. If the token is an operator, pop the right operand and then the left operand.

4. Perform the operation and push the result back onto the stack.

5. After all tokens are processed, the top of the stack is the final result.

PROGRAM (C)

```
#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>
#define MAX 100

int main(void) {
    char exp[MAX], *p;
    int st[MAX], top = -1, a, b;
    printf("Enter space-separated postfix expression: ");
    fgets(exp, sizeof exp, stdin);
    for (p = exp; *p; ) {
        char *end;
        long value;
        while (isspace((unsigned char)*p)) p++;
        if (*p == '\\0') break;
        if (isdigit((unsigned char)*p) ||
            ((*p == '-' || *p == '+') && isdigit((unsigned char)p[1]))) {
            value = strtol(p, &end, 10);
            st[++top] = (int)value;
            p = end;
        } else {
            char op = *p++;
            if (top < 1) { printf("Invalid expression\\n"); return 1; }
            b = st[top--]; a = st[top--];
            switch (op) {
            case '+': st[++top] = a + b; break;
            case '-': st[++top] = a - b; break;
            case '*': st[++top] = a * b; break;
            case '/':
                if (b == 0) { printf("Division by zero\\n"); return 1; }
                st[++top] = a / b; break;
            default: printf("Invalid operator\\n"); return 1;
            }
        }
    }
    if (top == 0) printf("Result = %d\\n", st[top]);
    else printf("Invalid expression\\n");
    return 0;
}
```

SAMPLE OUTPUT

Input: 5 3 2 * +. Output: Result = 11

RESULT

Thus, the postfix expression was evaluated successfully using a stack.

EXPERIMENT 6: Simple Queue

AIM

To implement insertion, deletion and display operations in a linear queue using an array.

ALGORITHM

1. Initialize FRONT and REAR to -1.

2. For ENQUEUE, check whether REAR is at the last array position. If empty, set FRONT to 0; increment REAR and insert the item.

3. For DEQUEUE, check whether the queue is empty. Remove the FRONT item and increment FRONT.

4. If FRONT becomes greater than REAR, reset both to -1.

5. Display elements from FRONT to REAR and repeat the menu.

PROGRAM (C)

```
#include <stdio.h>
#define MAX 50

int main(void) {
    int q[MAX], front = -1, rear = -1, ch, x, i;
    do {
        printf("\\n1.Enqueue 2.Dequeue 3.Display 4.Exit\\n");
        scanf("%d", &ch);
        if (ch == 1) {
            if (rear == MAX - 1) printf("Queue overflow\\n");
            else {
                printf("Enter value: "); scanf("%d", &x);
                if (front == -1) front = 0;
                q[++rear] = x;
            }
        } else if (ch == 2) {
            if (front == -1) printf("Queue underflow\\n");
            else {
                printf("Deleted: %d\\n", q[front++]);
                if (front > rear) front = rear = -1;
            }
        } else if (ch == 3) {
            if (front == -1) printf("Queue is empty\\n");
            else { for (i = front; i <= rear; i++) printf("%d ", q[i]); printf("\\n"); }
        }
    } while (ch != 4);
    return 0;
}
```

SAMPLE OUTPUT

Example: ENQUEUE 10, ENQUEUE 20, DISPLAY. Output: 10 20. DEQUEUE displays Deleted: 10.

RESULT

Thus, enqueue, dequeue and display operations were performed successfully in a simple queue.

EXPERIMENT 7: Priority Queue

AIM

To implement a priority queue in which the element with the highest priority (smallest priority number) is deleted first.

ALGORITHM

1. Store each item with its priority number in an array.

2. For insertion, add the item and its priority if the array has space.

3. For deletion, find the element with the smallest priority number.

4. Display that element, remove it by shifting the remaining entries and reduce the count.

5. Repeat the menu until exit.

PROGRAM (C)

```
#include <stdio.h>
#define MAX 50

struct Item { int data, priority; };

int main(void) {
    struct Item q[MAX], temp;
    int n = 0, ch, i, best;
    do {
        printf("\\n1.Insert 2.Delete 3.Display 4.Exit\\n");
        scanf("%d", &ch);
        if (ch == 1) {
            if (n == MAX) printf("Queue is full\\n");
            else {
                printf("Enter data and priority (smaller number = higher priority): ");
                scanf("%d%d", &q[n].data, &q[n].priority);
                n++;
            }
        } else if (ch == 2) {
            if (n == 0) printf("Queue is empty\\n");
            else {
                best = 0;
                for (i = 1; i < n; i++)
                    if (q[i].priority < q[best].priority) best = i;
                printf("Deleted: %d\\n", q[best].data);
                for (i = best; i < n - 1; i++) q[i] = q[i + 1];
                n--;
            }
        } else if (ch == 3) {
            if (n == 0) printf("Queue is empty\\n");
            for (i = 0; i < n; i++)
                printf("Data=%d Priority=%d\\n", q[i].data, q[i].priority);
        }
    } while (ch != 4);
    return 0;
}
```

SAMPLE OUTPUT

Insert (A,2), (B,1), (C,3). Delete output: B, because priority 1 is highest.

RESULT

Thus, insertion, priority-based deletion and display operations were implemented successfully.

EXPERIMENT 8: Singly Linked List

AIM

To create a singly linked list and perform insertion at the end, deletion by value and traversal.

ALGORITHM

1. Define a node with data and a pointer to the next node.
2. Initially set HEAD to NULL.
3. For insertion, create a node. If the list is empty make it HEAD; otherwise link it after the last node.
4. For deletion, search for the value while keeping track of the previous node, then adjust links and free the node.
5. For traversal, start at HEAD and print nodes until NULL.
6. Repeat the menu and free remaining nodes before exit.

PROGRAM (C)

```
#include <stdio.h>
#include <stdlib.h>

struct Node { int data; struct Node *next; };

int main(void) {
    struct Node *head = NULL, *tail, *p, *prev, *node;
    int ch, x;
    do {
        printf("\\n1.Insert end 2.Delete value 3.Display 4.Exit\\n");
        scanf("%d", &ch);
        if (ch == 1) {
            printf("Enter value: "); scanf("%d", &x);
            node = malloc(sizeof *node);
            if (!node) return 1;
            node->data = x; node->next = NULL;
            if (!head) head = node;
            else { for (tail = head; tail->next; tail = tail->next) {} tail->next = node; }
        } else if (ch == 2) {
            printf("Enter value: "); scanf("%d", &x);
            p = head; prev = NULL;
            while (p && p->data != x) { prev = p; p = p->next; }
            if (!p) printf("Value not found\\n");
            else {
                if (!prev) head = p->next; else prev->next = p->next;
                free(p); printf("Deleted\\n");
            }
        } else if (ch == 3) {
            for (p = head; p; p = p->next) printf("%d -> ", p->data);
            printf("NULL\\n");
        }
    } while (ch != 4);
    while (head) { p = head; head = head->next; free(p); }
    return 0;
}
```

SAMPLE OUTPUT

Insert 10, 20, 30; display output: 10 -> 20 -> 30 -> NULL. Delete 20; display: 10 -> 30 -> NULL.

RESULT

Thus, insertion, deletion and traversal were implemented successfully using a singly linked list.

EXPERIMENT 9: Doubly Linked List

AIM

To create a doubly linked list and perform insertion at the end, deletion by value and forward traversal.

ALGORITHM

1. Define a node with data, previous pointer and next pointer.
2. Initially set HEAD and TAIL to NULL.
3. For insertion, create a node and connect it after TAIL; update HEAD when the list is empty.
4. For deletion, locate the value and reconnect its previous and next nodes. Update HEAD or TAIL when needed.
5. For traversal, move from HEAD through next pointers and display each node.
6. Free all nodes before stopping.

PROGRAM (C)

```
#include <stdio.h>
#include <stdlib.h>

struct Node { int data; struct Node *prev, *next; };

int main(void) {
    struct Node *head = NULL, *tail = NULL, *p, *node;
    int ch, x;
    do {
        printf("\\n1.Insert end 2.Delete value 3.Display 4.Exit\\n");
        scanf("%d", &ch);
        if (ch == 1) {
            printf("Enter value: "); scanf("%d", &x);
            node = malloc(sizeof *node);
            if (!node) return 1;
            node->data = x; node->prev = tail; node->next = NULL;
            if (tail) tail->next = node; else head = node;
            tail = node;
        } else if (ch == 2) {
            printf("Enter value: "); scanf("%d", &x);
            for (p = head; p && p->data != x; p = p->next) {}
            if (!p) printf("Value not found\\n");
            else {
                if (p->prev) p->prev->next = p->next; else head = p->next;
                if (p->next) p->next->prev = p->prev; else tail = p->prev;
                free(p); printf("Deleted\\n");
            }
        } else if (ch == 3) {
            for (p = head; p; p = p->next) printf("%d <-> ", p->data);
            printf("NULL\\n");
        }
    } while (ch != 4);
    while (head) { p = head; head = head->next; free(p); }
    return 0;
}
```

SAMPLE OUTPUT

Insert 10, 20, 30; display output: 10 <-> 20 <-> 30 <-> NULL. Delete 20; display: 10 <-> 30 <-> NULL.

RESULT

Thus, insertion, deletion and traversal were implemented successfully using a doubly linked list.

EXPERIMENT 10: Circular Queue

AIM

To implement enqueue, dequeue and display operations using an array-based circular queue.

ALGORITHM

1. Initialize FRONT and REAR to -1.

2. The queue is full when (REAR + 1) % MAX equals FRONT.

3. For ENQUEUE, if empty set FRONT to 0; update REAR using (REAR + 1) % MAX and insert the item.

4. For DEQUEUE, remove the FRONT item. If it was the only item, reset both pointers; otherwise update FRONT circularly.

5. Display elements from FRONT until REAR, wrapping around using modulo MAX.

PROGRAM (C)

```
#include <stdio.h>
#define MAX 5

int main(void) {
    int q[MAX], front = -1, rear = -1, ch, x, i;
    do {
        printf("\\n1.Enqueue 2.Dequeue 3.Display 4.Exit\\n");
        scanf("%d", &ch);
        if (ch == 1) {
            if ((rear + 1) % MAX == front) printf("Queue is full\\n");
            else {
                printf("Enter value: "); scanf("%d", &x);
                if (front == -1) front = 0;
                rear = (rear + 1) % MAX; q[rear] = x;
            }
        } else if (ch == 2) {
            if (front == -1) printf("Queue is empty\\n");
            else {
                printf("Deleted: %d\\n", q[front]);
                if (front == rear) front = rear = -1;
                else front = (front + 1) % MAX;
            }
        } else if (ch == 3) {
            if (front == -1) printf("Queue is empty\\n");
            else {
                i = front;
                while (1) {
                    printf("%d ", q[i]);
                    if (i == rear) break;
                    i = (i + 1) % MAX;
                }
                printf("\\n");
            }
        }
    } while (ch != 4);
    return 0;
}
```

SAMPLE OUTPUT

Enqueue 10, 20, 30; display output: 10 20 30. Dequeue output: Deleted: 10.

RESULT

Thus, circular queue operations were implemented successfully using modulo arithmetic.

EXPERIMENT 11: Tree Traversal

AIM

To create a binary search tree and display its elements using inorder, preorder and postorder traversals.

ALGORITHM

1. Define a binary tree node containing data, left pointer and right pointer.

2. Insert each value into the binary search tree: smaller values go left and larger values go right.

3. Inorder traversal: visit left subtree, root, then right subtree.

4. Preorder traversal: visit root, left subtree, then right subtree.

5. Postorder traversal: visit left subtree, right subtree, then root.

6. Display all three traversals.

PROGRAM (C)

```
#include <stdio.h>
#include <stdlib.h>

struct Node { int data; struct Node *left, *right; };

struct Node *insert(struct Node *root, int x) {
    if (!root) {
        root = malloc(sizeof *root);
        if (!root) exit(1);
        root->data = x; root->left = root->right = NULL;
    } else if (x < root->data) root->left = insert(root->left, x);
    else if (x > root->data) root->right = insert(root->right, x);
    return root;
}
void inorder(struct Node *r) {
    if (r) { inorder(r->left); printf("%d ", r->data); inorder(r->right); }
}
void preorder(struct Node *r) {
    if (r) { printf("%d ", r->data); preorder(r->left); preorder(r->right); }
}
void postorder(struct Node *r) {
    if (r) { postorder(r->left); postorder(r->right); printf("%d ", r->data); }
}
void freeTree(struct Node *r) {
    if (r) { freeTree(r->left); freeTree(r->right); free(r); }
}
int main(void) {
    struct Node *root = NULL;
    int n, x, i;
    printf("Enter number of nodes: "); scanf("%d", &n);
    printf("Enter values: ");
    for (i = 0; i < n; i++) { scanf("%d", &x); root = insert(root, x); }
    printf("Inorder: "); inorder(root);
    printf("\\nPreorder: "); preorder(root);
    printf("\\nPostorder: "); postorder(root);
    printf("\\n");
    freeTree(root);
    return 0;
}
```

SAMPLE OUTPUT

Input: 5 3 7 2 4. Inorder: 2 3 4 5 7; Preorder: 5 3 2 4 7; Postorder: 2 4 3 7 5.

RESULT

Thus, inorder, preorder and postorder traversals of a binary search tree were implemented successfully.

EXPERIMENT 12: Graph Search Algorithms (DFS and BFS)

AIM

To represent an undirected graph using an adjacency matrix and traverse it using DFS and BFS.

ALGORITHM

1. Read the number of vertices and the adjacency matrix.

2. DFS: mark the current vertex as visited, display it, and recursively visit each unvisited adjacent vertex.

3. BFS: initialize an empty queue, mark and enqueue the start vertex.

4. While the queue is not empty, dequeue and display a vertex; enqueue each unvisited adjacent vertex and mark it visited.

5. Run DFS and BFS from the chosen starting vertex.

PROGRAM (C)

```
#include <stdio.h>
#define MAX 20

int graph[MAX][MAX], visited[MAX], n;

void dfs(int v) {
    int i;
    visited[v] = 1;
    printf("%d ", v);
    for (i = 0; i < n; i++)
        if (graph[v][i] && !visited[i]) dfs(i);
}
void bfs(int start) {
    int q[MAX], front = 0, rear = 0, i, v;
    for (i = 0; i < n; i++) visited[i] = 0;
    visited[start] = 1; q[rear++] = start;
    while (front < rear) {
        v = q[front++]; printf("%d ", v);
        for (i = 0; i < n; i++)
            if (graph[v][i] && !visited[i]) {
                visited[i] = 1; q[rear++] = i;
            }
    }
}
int main(void) {
    int i, j, start;
    printf("Enter number of vertices: "); scanf("%d", &n);
    printf("Enter adjacency matrix:\\n");
    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++) scanf("%d", &graph[i][j]);
    printf("Enter starting vertex (0 to %d): ", n - 1); scanf("%d", &start);
    for (i = 0; i < n; i++) visited[i] = 0;
    printf("DFS: "); dfs(start);
    printf("\\nBFS: "); bfs(start);
    printf("\\n");
    return 0;
}
```

SAMPLE OUTPUT

For edges 0-1, 0-2, 1-3, 1-4 and start vertex 0: DFS: 0 1 3 4 2; BFS: 0 1 2 3 4 (adjacency order).

RESULT

Thus, graph traversal was performed successfully using Depth First Search and Breadth First Search.

EXPERIMENT 13: Bubble, Selection and Heap Sort

AIM

To sort a list of integers in ascending order using bubble sort, selection sort and heap sort.

ALGORITHM

1. Read n elements into an array.

2. Bubble sort: repeatedly compare adjacent elements and swap them if they are in the wrong order.

3. Selection sort: find the smallest element in the unsorted portion and swap it with the first unsorted element.

4. Heap sort: build a max heap, repeatedly move the root (maximum) to the end and restore the heap.

5. Display the sorted array produced by each algorithm.

PROGRAM (C)

```
#include <stdio.h>
#define MAX 100

void print(int a[], int n) {
    int i;
    for (i = 0; i < n; i++) printf("%d ", a[i]);
    printf("\\n");
}
void copy(int src[], int dst[], int n) {
    int i; for (i = 0; i < n; i++) dst[i] = src[i];
}
void bubble(int a[], int n) {
    int i, j, t;
    for (i = 0; i < n - 1; i++)
        for (j = 0; j < n - i - 1; j++)
            if (a[j] > a[j + 1]) {
                t = a[j]; a[j] = a[j + 1]; a[j + 1] = t;
            }
}
void selection(int a[], int n) {
    int i, j, min, t;
    for (i = 0; i < n - 1; i++) {
        min = i;
        for (j = i + 1; j < n; j++)
            if (a[j] < a[min]) min = j;
        t = a[i]; a[i] = a[min]; a[min] = t;
    }
}
void heapify(int a[], int n, int i) {
    int largest = i, l = 2 * i + 1, r = 2 * i + 2, t;
    if (l < n && a[l] > a[largest]) largest = l;
    if (r < n && a[r] > a[largest]) largest = r;
    if (largest != i) {
        t = a[i]; a[i] = a[largest]; a[largest] = t;
        heapify(a, n, largest);
    }
}
void heapSort(int a[], int n) {
    int i, t;
    for (i = n / 2 - 1; i >= 0; i--) heapify(a, n, i);
    for (i = n - 1; i > 0; i--) {
        t = a[0]; a[0] = a[i]; a[i] = t;
        heapify(a, i, 0);
    }
}
int main(void) {
    int original[MAX], a[MAX], b[MAX], c[MAX], n, i;
    printf("Enter number of elements: "); scanf("%d", &n);
    printf("Enter elements: ");
    for (i = 0; i < n; i++) scanf("%d", &original[i]);
    copy(original, a, n); bubble(a, n);
    copy(original, b, n); selection(b, n);
    copy(original, c, n); heapSort(c, n);
    printf("Bubble sort: "); print(a, n);
    printf("Selection sort: "); print(b, n);
    printf("Heap sort: "); print(c, n);
    return 0;
}
```

SAMPLE OUTPUT

Input: 5 4 1 3 2. Bubble sort: 1 2 3 4 5; Selection sort: 1 2 3 4 5; Heap sort: 1 2 3 4 5.

RESULT

Thus, bubble sort, selection sort and heap sort were implemented and all produced the elements in ascending order.